

Formal Verification of the Lightning Network in TAMARIN

Abstract

The Lightning Network routes payments across chains of untrusted intermediaries using hashed time-locked contracts (HTLCs) layered on top of a two-party channel revocation mechanism. Prior symbolic analyses have treated these layers separately: two-party HTLC constructions on one side, generic message-forwarding abstractions on the other. Their interaction — revocation and punishment together with multi-hop routing, per-hop fees, and staggered block-height timing — has not been mechanised as a single symbolic model.

We present such a model in TAMARIN, structured as five theories that together discharge 87 lemma checks over 61 distinct properties under a full Dolev–Yao adversary. Building it surfaced a modelling pitfall we believe generalises beyond Lightning: encoding a one-shot on-chain output as a persistent fact permits the corresponding rule to fire without bound, and every individual safety lemma continues to hold while it does. We found two independent instances of this in our own model — unbounded punishment for a single act of cheating, and unbounded HTLC reuse of one channel output — diagnosed both as the same defect, and repaired both with linear single-spend tokens. We state the repair as an auditable discipline.

We further prove value conservation with per-hop fees, establish empirically that the intermediary claim obligation is not implied by the timelock ordering constraints, and reproduce the known wormhole attack. Because our model carries amounts rather than abstracting them away, we pin the wormhole’s theft to exactly the sum of the bypassed fees — a quantification the underlying formalism must carry numbers to express. Finally, we lift the safety results to paths of unbounded length by abstracting the routing layer, and give the refinement argument that justifies the transfer.

1 Introduction

Bitcoin [17] is the most important cryptocurrency in terms of market capitalisation. It is, however, lacking in many usability and scalability aspects: it is costly, has low bandwidth (only a few dozen transactions per second), and high latency. Many protocols strive to alleviate those shortcomings.

Payment Channel Networks. Payment channels enable a pair of users to quickly and cheaply perform

arbitrarily many payments by relying on the blockchain only to resolve disputes. Both agents pool together funds and then iteratively update their balance. The scripts involved and the protocol itself ensure that no honest agent loses money, by relying on a revocation mechanism: any party closing a channel using an outdated favourable state can be punished, consequently losing all funds invested in the channel.

Payment channels are, however, constrained to pairs of users. Two unconnected users still need to establish a channel in order to perform a payment, which is often not cost effective. Thus arose the concept of payment channel networks: one transitively propagates funds across payment channels to perform a payment between two users that are part of the same network but not connected directly. The most popular such protocol is perhaps the Lightning Network [18], with about \$170M locked on the network [1].

Due to the sheer complexity of its specification [12], it suffers from many attacks [2, 14, 20]. Despite promising ongoing efforts [10, 9] it does not yet feature a complete and unbounded verified formalisation. This paper attempts to alleviate that issue.

Protocol Verification. Designing and analysing cryptographic protocols is a notoriously difficult and error-prone task, as already illustrated by the various attacks and tentative fixes on the Lightning Network. Formal methods provide hope for more robust verification, and for automation.

Protocol verification is split between two main classes of models: the computational model, inherited from cryptography, where attackers and participants are polynomial-time algorithms and results are typically probabilistic; and the symbolic or Dolev–Yao model [8], where the attacker has infinite computational power but is limited to a term algebra. The symbolic model abstracts away from cryptography and focuses on the logic of a protocol, where much of the attacker activity is typically found.

Although tools exist for the computational model — CRYPTOVERIF [6], EASYCRYPT [4], SQUIRREL [3], CRYPTOVAMPIRE [11] — they are brittle, hard to use, or narrowly scoped. Symbolic verification, by contrast, enjoys successful automated verifiers in PROVERIF [5] and TAMARIN [16]. TAMARIN is a multi-set rewriting-based automated protocol verifier with a pedigree of widely used protocols verified, including WireGuard [13] and TLS 1.3 [7].

Contributions. We provide a modelisation of the Lightning Network in TAMARIN featuring unbounded channels and routing on unbounded paths. Specifically:

- **An integrated model.** We mechanise the two-party channel lifecycle — including revocation and punishment — together with multi-hop HTLC routing, per-hop fees, and staggered CLTV timing, in one symbolic model under a full Dolev–Yao adversary. To our knowledge this combination has not previously been machine-checked. The model is modular: five theories establishing 61 distinct properties across 87 lemma checks.
- **A linear single-spend discipline.** We identify a modelling pitfall in which a one-shot on-chain output is encoded as a persistent fact, permitting the corresponding rule to fire without bound while every safety lemma continues to hold. We found two independent instances in our own model and repaired both uniformly. The lesson generalises to any multiset-rewriting model of a blockchain protocol.
- **Quantified economic properties.** We prove end-to-end value conservation with strictly positive per-hop fees, and use it to pin the wormhole attack’s theft to exactly the sum of the bypassed fees rather than to unspecified “loss”.
- **Empirical assumption minimality.** We show by counterexample that the intermediary claim obligation is not implied by the timelock ordering constraints, and that the early-timeout race is reachable precisely when its ordering assumption is dropped.
- **Unbounded path length.** We lift the safety results to paths of arbitrary length by abstracting the routing layer, and give the refinement argument that justifies the transfer, along with an explicit account of what the abstraction costs.

We do *not* model payment privacy or unlinkability, which are confidentiality properties requiring onion routing and observational-equivalence proofs. We state this as an explicit exclusion rather than an oversight.

2 Related Work

Formal verification of payment channels. Litos et al. [10] give a formally verified Lightning implementation in WHY3, proving functional correctness of a single channel. Their treatment is deeper than ours at the channel layer and absent at the routing layer: there is no multi-hop path, no fee arithmetic, and no adversarial network model. Symbolic analyses in TAMARIN have targeted two-party HTLC constructions such as cross-chain swaps and contingent payments; these

lack multi-hop routing, lifecycle revocation, and staggered CLTV arithmetic.

Model checking. Grundmann and Hartenstein [9] verify balance security for Lightning channels with TLA⁺, using a stepwise refinement. Model checking buys them a faithful treatment of concurrency that a symbolic prover does not naturally provide; the cost is that their analysis is bounded — in the number of sessions, the depth of the state space, and the path length. Our analysis is unbounded in sessions and in adversary derivation, and, at the abstract layer, in path length. The two approaches are complementary rather than competing.

Agent skipping and path integrity. Generic symbolic frameworks have studied path integrity and agent-skipping in message-forwarding protocols [19]. That line of work establishes that an intermediary can be bypassed, and does so at a level of generality that covers Lightning as an instance. Their formalism, however, carries no numeric sort: it can express *that* an agent was skipped but not *what* the skip cost. Our contribution at this point is not the attack, which is known [14], but its quantification (Theorem 5).

Attacks on Lightning. Malavolta et al. [14] introduced the wormhole attack and proposed anonymous multi-hop locks as a countermeasure. Tikhomirov et al. [20] give a quantitative analysis of security, anonymity and scalability. Aumayr et al. [2] construct generalised channels from limited scripts. We reproduce the wormhole as a fidelity check on our model rather than as a new finding.

3 Protocol Overview and Threat Model

3.1 Payment Channel Lifecycle

In a payment channel, two parties broadcast an on-chain funding transaction. Once the channel is open, they transact off-chain by exchanging signed commitments, and update the state by exchanging new ones. When the state is updated, the parties exchange revocation secrets for the previous commitment and revoke the old state. If one party publishes an old commitment after revocation, the other may claim all funds. Between state updates, any commitment that has not yet been revoked is a valid unilateral close: if either party publishes one there is no penalty. Punishment is triggered only when a revoked commitment is published.

Channels close either cooperatively, with both parties agreeing on the final balance and settling in a single chain transaction, or unilaterally. In a unilateral close one party broadcasts an unrevoked commitment,

and both must wait for a relative timelock (CSV) to expire. That delay is what gives the counterparty time to detect whether the published state was revoked. If it was, the counterparty punishes the cheater and claims the entirety of the channel funds.

3.2 Multi-Hop HTLC Routing

When sender and recipient have no direct channel, the payment is routed through intermediaries along a path $S \rightarrow F_1 \rightarrow \dots \rightarrow F_n \rightarrow R$. Because intermediaries are untrusted, each hop is conditioned on a single shared secret, so that the payment either completes along the entire path or completes nowhere.

Lock round. The receiver generates a preimage x and sends the sender an invoice containing the hash $y = H(x)$ and the requested amount v , keeping x secret. The sender adds an HTLC to its channel with F_1 , locking funds conditional on y before a deadline d_0 ; after d_0 the payer may reclaim them. The unlock condition is to present x' with $H(x') = y$ before the deadline. Each forwarder in turn locks an HTLC on its outgoing channel under the *same* hash, deducting a routing fee, so that amounts decrease and deadlines tighten toward the receiver:

$$v_0 > v_1 > \dots > v_n = v, \quad d_0 > d_1 > \dots > d_n.$$

No funds move during this round. Each HTLC becomes an additional conditional output in the two parties' current commitment transaction, and is therefore subject to the revocation mechanism of Sec. 3.1. Locking is safe for F_i precisely because it holds an incoming HTLC on the same hash for a strictly larger amount: if its outgoing HTLC is claimed, the secret needed to claim its incoming one is thereby exposed.

Release round. The receiver reveals x to the last forwarder to claim the final HTLC. Since every HTLC shares the preimage, each intermediary claims on its incoming channel using the same x , learning it when the next hop claims downstream. To claim v_{n-1} from F_{n-1} , F_n presents x on its incoming HTLC and retains $v_{n-1} - v_n$ as its fee. The preimage propagates backward hop by hop until the sender's HTLC is settled. If any party fails to forward and the timelock expires, funds return to whoever locked them. If honest forwarders claim on time, everyone is paid; otherwise no one is.

Why deadlines must decrease. Consider F_i holding an incoming HTLC at d_{i-1} and an outgoing one at d_i . If F_{i+1} claims the outgoing HTLC at the latest permitted moment d_i , then F_i learns x at d_i and retains the window (d_i, d_{i-1}) in which to settle its incoming claim. The gap $d_{i-1} - d_i$ is the CLTV delta, and it is precisely the time reserved for F_i to recover its funds.

Were the ordering inverted, so that $d_{i-1} < d_i$, then F_i 's incoming HTLC would expire and be refunded before the downstream party claims at d_i : F_i would have paid v_i downstream while recovering nothing upstream, losing the full forwarded amount. Sec. 6.3 derives the positivity of this delta and the non-emptiness of the resulting window from a concrete block clock, rather than assuming them.

Residual liveness requirement. A non-empty claim window is necessary but not sufficient: nothing in the timelock structure compels F_i to use it. An intermediary offline during (d_i, d_{i-1}) forfeits its incoming HTLC even though every party followed the protocol and no one cheated. Routing security therefore reduces, at this point, to a liveness assumption on honest intermediaries, which corresponds in deployment to watchtower infrastructure [15].

Furthermore, the entire release round depends on the receiver disclosing x . While funds are locked, every party on the path is waiting for one of two events:

- the receiver releases x , which propagates backward so that each party claims its HTLC and collects its fee; or
- the deadline d_i passes, the HTLC times out, and the party recovers its funds.

No one can force the receiver to reveal x . A receiver that simply declines immobilises the capital of every party on the path until the deadlines elapse — a grieving denial of service. Separately, the *wormhole attack* occurs when two dishonest, non-adjacent parties communicate directly, relaying x between themselves and skipping the intermediaries in between without disclosing it to them. The skipped intermediary is honest throughout: it forwards value and earns no fee. Both pathologies are reachable in our model.

3.3 Threat Model and Security Goals

We assume the standard Dolev–Yao adversary model [8]. The adversary controls the whole network: it can read, modify, and inject messages. Its only limitation is that it cannot break cryptographic primitives. We additionally model long-term key compromise as a separate adversarial capability, so that properties can be stated conditionally on honesty rather than assuming it globally.

Liveness assumptions. Security at the routing layer degrades into a liveness requirement: a node that goes offline loses funds even in an otherwise correct execution. We encode this with four timing restrictions, distinguishing passive orderings from active obligations.

- **Structural clock constraints.** T1 (`RedeemBeforeTimeout`), T2a (`ForwardTimeGap`), and T2b (`HonestPartiesActBeforeIncomingTimeout`) are assumed structural constraints of the CLTV clock.
- **Active claiming.** T3 (`IntermediaryMustClaim`) requires an intermediary to act: it must claim its incoming HTLC once its outgoing HTLC is redeemed.

T1, T2a and T2b carry timing soundness; T3 carries intermediary safety. Sec. 6.3 shows that none is implied by the others. Our security goals translate directly into verified lemmas: payment atomicity, intermediary safety, routing causality and authentication, timing safety, and value conservation.

4 Symbolic Model and Notation

We model the protocol in TAMARIN [16], a symbolic tool for security-protocol analysis. A system state S is a finite multiset of facts. Rules are transitions, enabled when their premises are present in the state. A rule consists of three sets of facts:

$$[L] - [A] \rightarrow [R] \quad (1)$$

where L are the premises, A the action facts, and R the conclusions. Firing a rule removes the linear premises from the state and adds the conclusions; action facts are not part of the state but are recorded on the trace.

TAMARIN provides built-in fact symbols `In()` and `Out()` representing the adversary-controlled network under Dolev–Yao [8], and `Fr()` for generating fresh, unique values. Facts are either *linear* or *persistent*. A linear fact is consumed when a rule fires; a persistent fact, written with a leading bang as $!F(t)$, is checked but never removed, and may license unboundedly many rule firings. This distinction is the subject of Sec. 6.1 and the source of our main modelling finding.

We use the hashing, signing, natural-numbers and multiset built-ins; Sec. 7 explains why two different arithmetic builtins are needed. An execution produces a trace of action facts, and security goals are stated as first-order formulas over traces, proved either for all traces (safety) or for a single witnessing trace (exists-trace).

5 Formal Modelling Methodology

5.1 The Curse of an All-in-One Theory

Our initial approach was to put everything into a single theory: the channel lifecycle, the HTLC routing logic,

the CLTV block-height arithmetic, and value conservation. It did not work. When the signature equational theory, natural-number induction, and an unbounded block clock are combined, TAMARIN’s search space explodes and the prover exhausts available memory.

This was tested rather than assumed. Any *two* of the three ingredients remain tractable; only the full triple OOM-kills the prover. The model is therefore split so that no single theory carries all three. The resulting architecture is summarised in Table 1.

5.2 Tactic Engineering and Reproduction

All results were produced under a strict test-first discipline: a property is claimed only after TAMARIN reports it. Per-lemma invocation is used on the large theory to avoid OOM. The flag `--stop-on-trace=seqdfs` rescues the heavy existence witnesses but hangs the natural-number arithmetic theories, so it is applied selectively. All results were reproduced in a single sitting on one machine under TAMARIN 1.10.0 with Maude 3.1; every theory passes TAMARIN’s well-formedness checks, and the whole package verifies in 542.36 s of wall-clock time.

6 Verification of the Core Model

Across five theories (Table 1) we discharge 87 lemma checks over 61 distinct properties. The concrete theory `multihop.spthy` contributes 43 lemmas: 33 all-traces and 10 exists-trace witnesses, of which 5 all-traces lemmas are `[reuse]` helpers rather than protocol goals. The N -hop theory `Modif.spthy` re-proves 26 of these over an abstract routing layer and adds 5 new ones, for 31 in total. The remaining 13 come from the three timing theories.

Fig. 1 shows the flow the concrete theory models, with each stage annotated by the lemmas that constrain it.

6.1 Linear Facts and the Single-Spend Discipline

The central modelling lesson of this work concerns the choice between linear and persistent facts. Recall from Sec. 4 that a persistent fact $!F(t)$ is checked by a rule but not consumed, whereas a linear fact $F(t)$ is removed from the state when the rule fires. This distinction is standard; what is not obvious is which side of it an on-chain output belongs on.

Definition 1 (Single-spend discipline). *A protocol resource that corresponds to one on-chain output, spendable at most once, must be modelled by a linear fact. A resource that corresponds to knowledge, which*

Table 1: The five-theory modular architecture. The first four theories establish 56 distinct properties, and the N -hop theory adds a further 5, for 61 in total. Of the N -hop theory’s 31 lemmas, 26 restate properties already proved in `multihop.spthy` over the abstract routing layer and 5 are new to it, giving 87 lemma checks in total.

Theory File	Aspect Modelled	Safety	Witness	Time	Builtins
<code>multihop.spthy</code>	Lifecycle, 3-hop routing, value/fees, wormhole	33	10	353.31 s	Hashing, Signing, Nat
<code>Clock.spthy</code>	Block-clock lifecycle & staggered-CLTV claim windows	6	3	54.37 s	Natural-numbers
<code>Cltv.spthy</code>	Pure CLTV block arithmetic (delta positivity)	3	0	0.23 s	Natural-numbers
<code>timeout.spthy</code>	Early-timeout race (T2b removed)	0	1	0.06 s	—
<i>Distinct properties</i>		42	14		
<code>Modif.spthy</code>	<i>Re-proof</i> over the N -hop abstraction, plus fee/amount chaining	25	6	134.39 s	Hashing, Signing, Multiset
<i>Grand total (all five theories, 87 lemma checks)</i>		67	20	542.36 s	

cannot be un-learned, may be modelled by a persistent fact. Where a single event creates both — a secret becomes known and an output becomes spendable — the two must be separated into distinct facts.

Lightning’s revocation mechanism is exactly such an event, and it is a trap. When a channel state is revoked, the counterparty learns the revocation secret (knowledge, persistent) and simultaneously gains the right to spend one revoked commitment output (a resource, linear). The natural encoding conflates them, because the protocol description presents revocation as the disclosure of a secret, and disclosure is what persistent facts are for.

Instance 1: unbounded punishment. Our first formulation gated the punishment rule on the persistent revocation secret alone:

```
rule Publish_Revoked_A: // WRONG
[ !ChannelRecord(ptr, $A, $B), !ValidOnChain(ptr),
  !RevokedSecret($A, n_old, sAold, oldCommitA) ]
--[ PublishRevoked($A, n_old, oldCommitA), Cheat($A, n_old) ]->
[ OnChainCheat($A, $B, n_old, oldCommitA),
  RevealedSecret($A, n_old, sAold, oldCommitA) ]
```

Every premise here is persistent, so the rule may fire arbitrarily often for a single revoked state. The model admitted traces in which the honest party confiscates the channel balance repeatedly on the strength of one act of cheating. The defect is silent: every individual safety lemma about punishment continues to hold. `No_Punishment_Without_Cheating` is unaffected, since punishment still requires a prior cheat. Only the *multiplicity* is wrong, and no single safety lemma in the original formulation constrained multiplicity.

The repair separates the two roles. `Revoke_Old_Secret` now emits a linear token alongside the persistent secret:

```
[ StateA(...), StateB(...),
  !RevokedSecret($A, n_old, sAold, oldCommitA),
  !RevokedSecret($B, n_old, sBold, oldCommitB),
  // linear one-shot spend of the revoked commitment
  output
  RevokedCommitmentUnspent(ptr, $A, $B, n_old, oldCommitA),
  RevokedCommitmentUnspent(ptr, $B, $A, n_old, oldCommitB)
]
```

and `Publish_Revoked_A` consumes that token, so the

same commitment cannot enter the punishment path twice:

```
rule Publish_Revoked_A: // FIXED
[ !ChannelRecord(ptr, $A, $B), !ValidOnChain(ptr),
  !RevokedSecret($A, n_old, sAold, oldCommitA),
  RevokedCommitmentUnspent(ptr, $A, $B, n_old, oldCommitA)
]
--[ PublishRevoked($A, n_old, oldCommitA), Cheat($A, n_old) ]->
[ OnChainCheat($A, $B, n_old, oldCommitA),
  RevealedSecret($A, n_old, sAold, oldCommitA) ]
```

Knowledge of the secret is retained; only the spend is one-shot.

Instance 2: HTLC slot reuse. The same pathology appeared independently in the routing layer. A channel’s HTLC output was re-derivable from the persistent `!ChannelConnect` fact, so one channel could carry unboundedly many concurrent HTLCs on the same output. The repair is identical in shape: `Lock_Funds_And_Open` mints a linear `Free(ptr)` token, and every rule that adds an HTLC consumes it.

That two independent gaps in different layers had the same cause, and the same one-line repair, is what makes Definition 1 worth stating as a discipline rather than as two bug fixes.

Re-verification. Adding a linear premise shrinks a rule’s enabling condition, so $\text{traces}(R') \subseteq \text{traces}(R)$: every all-traces property is preserved automatically, but exists-trace witnesses may be destroyed. We therefore re-ran the full theory rather than appealing to monotonicity. All 43 lemmas re-verify. Notably the hardened N -hop theory verifies *faster* than the unhardened one (134 s versus 265 s), because each additional linear premise prunes the search space — the opposite of the intuition that had previously discouraged us from combining the two.

6.2 Protocol Properties

We state the main guarantees as theorems, each naming the TAMARIN lemma that discharges it. Throughout, “honest” abbreviates the absence of any long-term key compromise, which our model exposes as an explicit adversarial capability (`Compromise_Ltk`).

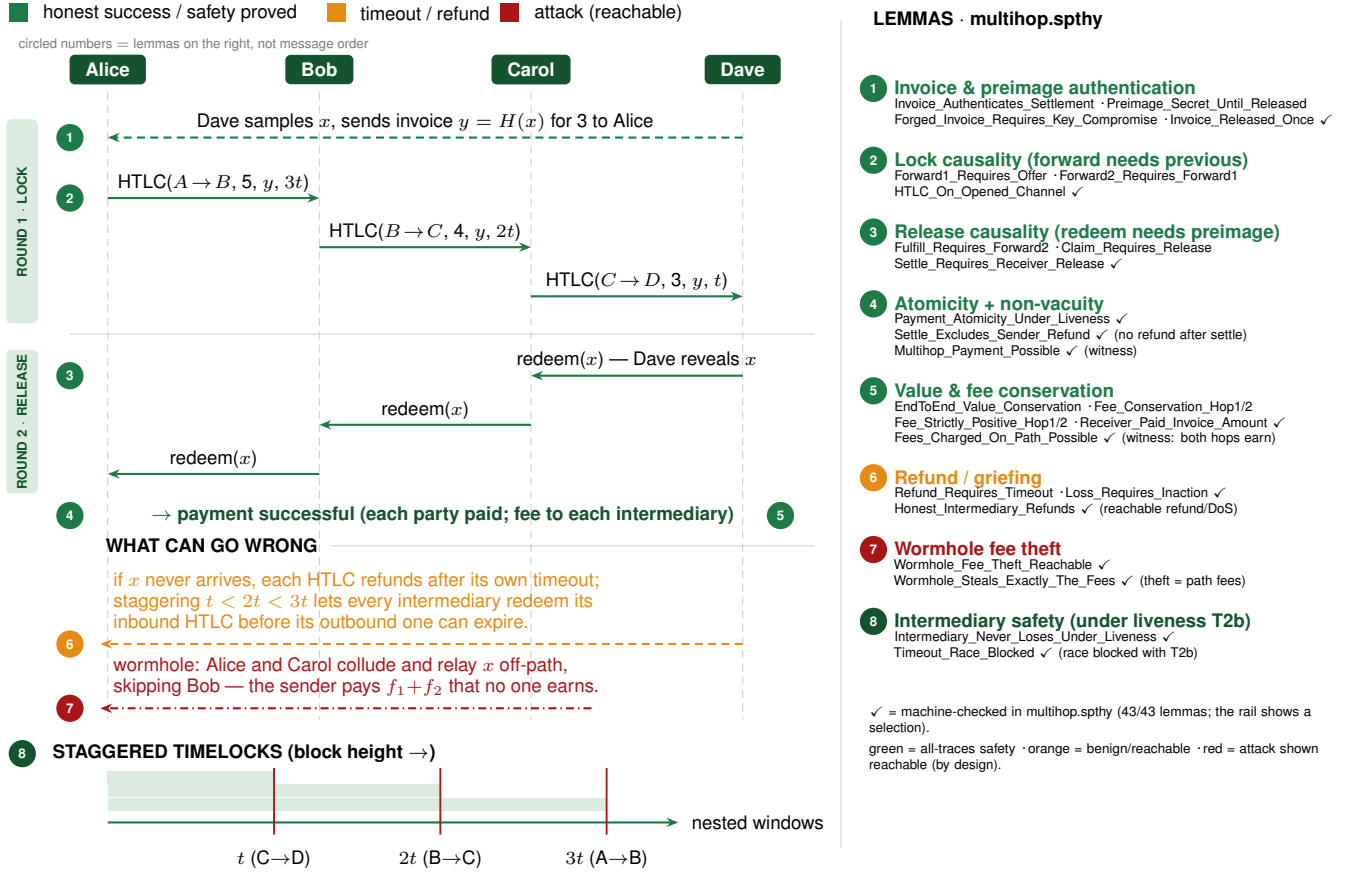


Figure 1: Multi-hop HTLC from Alice to Dave, annotated with lemmas from `multihop.spthy`. Circled numbers match the rail on the right (not message order). Green: proved safety; orange: reachable refund (grieving); red: reachable wormhole. Hop amounts $5 \rightarrow 4 \rightarrow 3$ are the fees.

Theorem 1 (Payment atomicity). *If both intermediaries forwarded, the receiver's HTLC was redeemed, and no key was compromised, then the sender's HTLC on the same hash cannot be refunded.* (*Payment_Atomicity_Under_Liveness*)

Theorem 2 (Intermediary safety). *If a forwarder's outgoing HTLC is redeemed and its incoming HTLC is refunded, then its long-term key was compromised.* (*Intermediary_Never_Loses_Under_Liveness*)

Theorem 2 is the contrapositive of the property that matters operationally: an honest forwarder never pays out downstream without recovering upstream. It holds only under the claim obligation T3, as Sec. 6.3 shows.

Theorem 3 (Routing causality). *Every forward is preceded by a genuine upstream offer or forward with agreeing amounts, unless the upstream party's key was compromised.* (*Forward1_Requires_Offer, Forward2_Requires_Forward1*)

The compromise-free versions are proved separately as `Forward1_Requires_Offer_Honest` and `For-`

`ward2_Requires_Forward1_Honest`, which drop the disjunct rather than carrying it.

Theorem 4 (Value conservation). *Over a complete linked three-hop payment, the sender locks strictly more than the receiver's invoice amount.* (*EndToEnd_Value_Conservation*)

The surplus is identified with the per-hop fees by `Fee_Conservation_Hop1` and `Fee_Conservation_Hop2`, each of which states $v_{\text{in}} = v_{\text{out}} + f$ at its hop. Taken together these give the quantitative statement the wormhole analysis needs: value is neither created nor destroyed along the path, and the difference between what the sender pays and what the receiver receives is exactly $\sum_i f_i$.

Non-vacuity. Safety properties are vacuously true of a model that cannot execute. We therefore accompany them with exists-trace witnesses. `Multihop_Payment_Possible` exhibits a complete eight-event payment from invoice to sender settlement. `Distinct_Parties_Configuration` exhibits three channels between four pairwise-distinct parties with no compromise and no dispute. `Fees_Charged_On_Path_Possible`

exhibits two *different* intermediaries each charging a fee on the same payment.

6.3 Timing Assumptions and Their Minimality

Sec. 3.3 introduced four timing restrictions. Two questions follow: are they sound, and is any of them redundant?

T2b is load-bearing. With T2b active, the early-timeout race is unreachable (`Timeout_Race_Blocked`). A minimal model with no such assumption exhibits the race as a concrete trace (`timeout.spthy`, `Early_Timeout_Race`). The pair forms a two-sided certificate: the race is reachable without the assumption and unreachable with it, so T2b is neither vacuous nor decorative.

T3 is not implied by T1 and T2. It is natural to ask whether the claim obligation T3 (`IntermediaryMustClaim`) follows from the ordering constraints. It does not. Removing T3 from `multihop.spthy` and re-running falsifies Theorem 2: TAMARIN returns a concrete 59-step counterexample. T2b orders the two timeouts, $\text{TimedOut}(\text{ptrOut}) < \text{TimedOut}(\text{ptrIn})$, but does not compel the node to *sweep* its incoming HTLC once the outgoing one is redeemed. That obligation is precisely what T3 encodes, and it corresponds in deployment to watchtower infrastructure [15].

Invariant 1 (Assumption minimality). *None of T1, T2a, T2b, T3 is implied by the others. T1–T2b carry timing soundness; T3 alone carries intermediary safety.*

Deriving what the core model assumes. The concrete theory treats time relationally, as an ordering of events. The two small natural-number theories discharge the arithmetic it takes for granted. `Cltv.spthy` derives that the CLTV delta is strictly positive and that the resulting claim window is non-empty across a staggered path. `Clock.spthy` runs a live block clock through the full forward lifecycle and derives the claim window between hops at concrete block heights (`Claim_Window_Exists`, `Transitive_Preimage_Before_Upstream_Deadline`), together with the lifecycle property that no HTLC may be added after a channel closes.

6.4 Griefing and the Wormhole Attack

Two pathologies survive an otherwise correct execution. Both are reachable in our model, and we distinguish sharply between them.

Griefing is an availability failure. `Honest_Intermediary_Refunds` exhibits a trace in which

capital is locked and later refunded with no settlement anywhere on the path and no key compromise. Every party follows the protocol; the receiver simply never discloses x . No safety property is violated — the funds return — but every forwarder’s liquidity is immobilised until its deadline elapses. This is a denial of service, not theft, and we report it as such.

The wormhole, quantified. The wormhole attack is known [14] and we claim no novelty for exhibiting it. Two non-adjacent colluding parties relay the preimage out of band, bypassing the honest intermediary between them, which forwards liquidity and earns nothing. `Wormhole_Fee_Theft_Reachable` finds this trace in 51 steps: the sender settles and the receiver is paid, yet the middle hop is never redeemed and neither intermediary records a `FeeEarned` event.

What is new is the quantification. Because our amounts are carried in the model rather than abstracted away, we can state and prove not merely that a loss occurs but exactly what it is.

Theorem 5 (Wormhole theft is exactly the path fees). *There is a trace in which the sender locks $v + f_1 + f_2$, the receiver is paid v , no key is compromised, and neither intermediary earns a fee. (`Wormhole_Steals_Exactly_The_Fees`, 81 steps)*

Prior symbolic treatments of agent-skipping establish that the honest intermediary is bypassed, but their formalisms carry no numeric sort and so cannot express *how much* is lost. Theorem 5 pins the adversary’s gain to $f_1 + f_2$, the sum of the bypassed fees, which is the economically meaningful statement.

7 Scaling to N Hops

The concrete theory fixes the path at three hops. Whether the same guarantees hold for paths of arbitrary length is the obvious next question, and it is not answerable by simply adding rules: each additional hop in the signed-message encoding multiplies the adversary’s options, and TAMARIN’s backward search does not terminate.

7.1 Abstracting the Routing Layer

The obstruction is the signed HTLC offer. Resolving where a forwarded message came from requires TAMARIN to consider every party as a possible origin, and with an unbounded number of hops that analysis recurses without bound. We therefore replace the network-level message with an internal linear fact handed directly from one hop to the next:

```
rule Forward_HTLC:
  [ Route($Prev, $Me, ptrIn, y, amtOut + fee),
```

```

!ChannelConnect(ptrOut, $Me, $Next),
Free(ptrOut) ]
--[ HTLCForwarded($Me, $Prev, $Next, ptrIn, ptrOut, y,
    amtOut + fee, amtOut),
    FeeExpected($Me, ptrIn, ptrOut, y, fee),
    Neq(ptrIn, ptrOut), Neq($Prev, $Me), Neq($Me, $Next)
    ]->
[ IntermediaryPending($Me, $Prev, $Next, ptrIn, ptrOut, y,
    amtOut + fee, amtOut, fee),
    Route($Me, $Next, ptrOut, y, amtOut) ]

```

This single rule fires at every hop, for any N . Because the HTLC is no longer a self-referential signed term, TAMARIN’s origin analysis no longer recurses. The price is that per-hop authentication is no longer machine-checked at this layer; Sec. 7.3 discharges that obligation separately.

7.2 Amount Chaining and the Choice of Builtin

The rule above is worth reading closely for one detail. The incoming amount is matched against the pattern `amtOut + fee`, where `+` is the associative-commutative operator of the `multiset` builtin. AC unification lets this pattern *split* the incoming amount nondeterministically: $\$Me$ keeps `fee` and locks `amtOut` downstream. The outgoing `Route` fact then carries `amtOut`, which becomes the next hop’s incoming amount. Value therefore chains along a path of any length.

This is why the two theories use different builtins. The `natural-numbers` builtin used in `multihop.spthy` provides `%+`, which can *check* a sum — `Eq(%vIn, %vOut %+ %fee)` — but cannot destructure one in a premise, because `%+` is not AC-unifiable. At three fixed hops, checking suffices, and `natural-numbers` keeps the arithmetic lemmas cheap. At unbounded N , splitting is required, so `multiset` is the only workable choice. The selection is driven by what each builtin can unify, not by preference.

Five properties are new at this layer and could not be stated at all without the chaining: `Forward_Requires_Incoming` (279 steps), which requires every forward to be preceded by an offer or a predecessor forward *with agreeing amounts*; `Amount_Strictly_Decreases_Per_Hop`; `Fee_Only_On_Successful_Forward`; `Re-deem_Requires_Receiver_Release`; and the witness `Multihop_Payment_With_Fees_Possible`.

7.3 Soundness of the Abstraction

The abstraction transfers safety from the concrete model to the abstract one only if the concrete model’s authentication guarantees justify it. We make this precise. Let C be the concrete theory and A the abstract one, and let π project a concrete trace onto the shared

alphabet Σ of routing events:

$$\Sigma = \left\{ \begin{array}{l} HTLCOffered, HTLCForwarded, \\ HTLCFulfilled, Redeemed, Refunded \end{array} \right\}.$$

Theorem 6 (Refinement). *Under the per-hop authentication guarantees of Theorem 3, for every honest trace t of C there is a trace t' of A with $\pi(t) = \pi(t')$. Consequently every Σ -property proved of A holds of C restricted to honest executions.*

The argument is by induction on the length of the concrete trace and is given in full in the artifact. The essential step is that Theorem 3 lets a concrete `Forward1-HTLC` step be matched by an abstract `Forward-HTLC` step consuming the corresponding `Route` fact: the signed message’s authenticity is exactly what the linear fact assumes.

Theorem 6 is a pen-and-paper result, not a machine-checked one, and we flag it as the principal soundness caveat of the N -hop extension.

Why the wormhole disappears. One consequence deserves comment. The wormhole is not reachable in the abstract model, and this is structural rather than accidental. In C the preimage travels as a network message that the adversary can relay out of band. In A the settlement chain is carried by linear `Fulfill(ptr, x)` facts, each consumed by exactly one downstream hop, so no party can be skipped. The abstraction that buys unbounded path length also removes the collusion side channel. Attacks that exist at the message layer must therefore be established in C ; only A carries them to arbitrary N . This is a limitation of the layering, and we state it rather than leaving it implicit.

Tractability boundary. One lemma, `Settle_Requires_Receiver_Release`, remains open at the abstract layer. Resolving `SenderSettled` backwards walks the entire generic settle chain, whose length is unbounded, and the search does not terminate. It verifies without difficulty in the fixed-hop theory. We report it as the single genuine tractability boundary of this layer rather than removing it from the file.

8 Discussion

What the discipline costs and what it buys. Adding linear tokens is not free: each one is a premise the prover must resolve, and each one narrows what the model admits. We found the cost to be negative in practice — the hardened N -hop theory verifies in half the time of the unhardened one — but the general point is that narrowing is exactly what makes the model faithful. A model that admits more traces than reality is not a more conservative model; it is a wrong one

that happens to fail safe on all-traces properties and fail open on witnesses.

Where the prover stops scaling. Three boundaries are worth recording. First, the triple of signing, natural-number induction and an unbounded clock is intractable together, which is why the package is modular rather than monolithic. Second, `Settle_Requires_Receiver_Release` does not terminate over the generic settle chain, though it verifies at fixed hop count. Third, the refinement argument of Theorem 6 is pen-and-paper. Each of these is a place where a reader should discount our claims, and we prefer to name them than to let them be discovered.

Threats to validity. Our model abstracts Bitcoin script: outputs are represented by pointers and spend conditions by rule premises, not by evaluated script. It treats signatures symbolically, so signature-level attacks are out of scope by construction. Timing in the concrete theory is relational, and the two block-clock theories that make it numeric do not carry the signing layer. Finally, the wormhole is unreachable in the abstract model for structural reasons (Sec. 7.3), so message-layer attacks must be established at the concrete layer and do not transfer to arbitrary N .

9 Conclusion and Future Work

We have given a modular, machine-checked model integrating the Lightning two-party channel lifecycle with multi-hop HTLC routing and fees, establishing 61 distinct properties across five theories. The integration surfaced a linear single-spend discipline: two independent soundness gaps arising from encoding one-shot on-chain outputs as persistent facts, repaired uniformly by linear tokens, and generalisable into an auditable modelling invariant. We established the minimality of the T3 claim obligation empirically, proved value conservation with strictly positive per-hop fees, exhibited a grieving denial of service, reproduced the wormhole attack and quantified its theft as exactly the bypassed fees, and mapped where the prover stops scaling.

Three directions follow. The refinement theorem should be machine-checked rather than argued on paper. The concrete and abstract layers should be reconciled so that message-layer attacks such as the wormhole can be stated at unbounded path length. And the channel layer would benefit from the numeric timing that currently lives only in the separate clock theories — which, on the evidence of Sec. 6.1, may become tractable once the model is sufficiently linear.

References

- [1] Lightning network statistics – 1ml, bitcoin mainnet, 2026. <https://1ml.com/statistics>.
- [2] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. Generalized channels from limited blockchain scripts and adaptor signatures. In *Advances in Cryptology – ASIACRYPT 2021*, volume 13091, pages 635–664. Springer, 2021.
- [3] David Baelde, Stéphanie Delaune, Charlie Jacomme, Adrien Koutsos, and Solène Moreau. An interactive prover for protocol verification in the computational model. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 537–554, 2021.
- [4] Gilles Barthe, Benjamin Grégoire, Sylvain Heraud, and Santiago Zanella Béguelin. Computer-aided security proofs for the working cryptographer. In *Advances in Cryptology – CRYPTO 2011*, pages 71–90. Springer, 2011.
- [5] Bruno Blanchet. An efficient cryptographic protocol verifier based on prolog rules. In *14th IEEE Computer Security Foundations Workshop (CSFW-14)*, pages 82–96, 2001.
- [6] Bruno Blanchet. A computationally sound mechanized prover for security protocols. In *2006 IEEE Symposium on Security and Privacy (S&P'06)*, pages 140–154, 2006.
- [7] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. A comprehensive symbolic analysis of TLS 1.3. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*, pages 1773–1788, 2017.
- [8] Danny Dolev and Andrew Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.
- [9] Matthias Grundmann and Hannes Hartenstein. Security of the lightning network: Model checking a stepwise refinement with TLA+. In *Integrated Formal Methods*, pages 313–335. Springer Nature Switzerland, 2026.
- [10] Grzegorz Grzegorz, Rafał Stefanski, and Orfeas Stefanos Thyfronitis Litos. A formally verified lightning network. In *Financial Cryptography and Data Security*, pages 3–20. Springer Nature Switzerland, 2026.

- [11] Simon Jeanteur, Laura Kovács, Matteo Maffei, and Michael Rawson. Cryptovampire: Automated reasoning for the complete symbolic attacker cryptographic model. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3165–3183, 2024.
- [12] Lightning Network. Lightning/bolts, 2026. <https://github.com/lightning/bolts>.
- [13] Benjamin Lipp, Bruno Blanchet, and Karthikeyan Bhargavan. A mechanised cryptographic proof of the wireguard virtual private network protocol. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 231–246, 2019.
- [14] Giulio Malavolta, Pedro Moreno-Sanchez, Clara Schneidewind, Aniket Kate, and Matteo Maffei. Anonymous multi-hop locks for blockchain scalability and interoperability. In *Proceedings 2019 Network and Distributed System Security Symposium (NDSS)*, 2019.
- [15] Patrick McCorry, Surya Bakshi, Iddo Bentov, Sarah Meiklejohn, and Andrew Miller. Pisa: Arbitration outsourcing for state channels. In *Proceedings of the 1st ACM Conference on Advances in Financial Technologies (AFT '19)*, pages 16–30, 2019.
- [16] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The TAMARIN prover for the symbolic analysis of security protocols. In *Computer Aided Verification*, pages 696–701. Springer, 2013.
- [17] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008.
- [18] Joseph Poon and Thaddeus Dryja. The bitcoin lightning network: Scalable off-chain instant payments, 2016.
- [19] Zach Smith, Peter Y. A. Ryan, et al. Modelling agent-skipping attacks in message forwarding protocols. In *Proceedings on Privacy Enhancing Technologies (POST)*, 2022. Bibliographic details need verification.
- [20] Sergei Tikhomirov, Pedro Moreno-Sanchez, and Matteo Maffei. A quantitative analysis of security, anonymity and scalability for the lightning network. In *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 387–396, 2020.